

PART XIV

GREEDY ALGORITHMS

Local choices, exchange arguments, and knowing when greedy fails

A greedy algorithm makes the choice that looks best right now, without reconsidering it later, and hopes that a sequence of locally-best choices adds up to a globally-best answer. Sometimes this hope is provably justified — Kruskal and Prim from the Graphs Part are both greedy algorithms that happen to be exactly correct. Sometimes it isn't, and greedy quietly gives a wrong answer that looks plausible. This Part is as much about proving greedy correct as it is about writing it — and, just as importantly, about recognizing the cases where it fails.

Contents of Part XIV

- 1. What Is a Greedy Algorithm?
- 2. Proving Greedy Correct: The Exchange Argument
- 3. Activity Selection and Interval Scheduling
- 4. Fractional Knapsack
- 5. Job Sequencing with Deadlines
- 6. Merging Problems and the Huffman Pattern
- 7. More Classic Greedy Patterns
- 8. When Greedy Fails
- 9. How to Recognize a Greedy Problem
- Part XIV Revision, Cheat Sheet, and Interview Questions

Chapter 1 — What Is a Greedy Algorithm?

In one sentence: a greedy algorithm builds a solution step by step, at each step picking whatever option looks best by some simple local rule, and never reconsidering that choice afterward.

This is the opposite instinct from dynamic programming (a later Part), which explores many possible choices and keeps whichever combination turns out best overall. Greedy is faster and simpler exactly because it refuses to look back — but that refusal is only safe when the problem has a specific structure that guarantees local optimality implies global optimality.

1.1 Local optimum vs. global optimum

A local optimum is the best choice available at the current step, given only what's known so far. A global optimum is the best possible outcome across the entire problem. Greedy always finds a local optimum at every step; the entire question this Part answers, problem by problem, is when that local optimum is guaranteed to combine into the global one.

Why this matters

There is no universal way to tell if greedy works for a brand-new problem just by trying it on a few examples — a greedy strategy can pass every small example you try by hand and still be wrong in general. The only reliable way to trust a greedy solution is an actual correctness argument, which is exactly what Chapter 2 builds.

Chapter 2 — Proving Greedy Correct: The Exchange Argument

In one sentence: an exchange argument shows that any optimal solution can be transformed, step by step, into the greedy solution without ever making it worse — which proves the greedy solution is at least as good as any optimal one.

The typical shape of an exchange argument:

- 1 Assume some optimal solution OPT exists that differs from the greedy solution.
- 2 Find the first point where OPT and greedy disagree.
- 3 Show that swapping OPT's choice at that point for greedy's choice does not make OPT any worse (often because greedy's choice is at least as good by the exact criterion the greedy rule uses).
- 4 Repeat this swap until OPT is identical to the greedy solution, without its quality ever decreasing.
- 5 Conclude: since OPT was optimal and is now equal to greedy's solution, greedy is also optimal.

This sounds abstract in general form; it becomes concrete and fairly short for each specific problem in this chapter. Chapter 3 below works through the full argument for activity selection as a template you can reuse.

⚠ Common mistake

Treating “it worked on the examples I tried” as a proof. Many wrong greedy strategies for interval and scheduling problems pass the first 2–3 examples someone tries by hand — always look for either an exchange argument or a matching counterexample before trusting a greedy idea.

Chapter 3 — Activity Selection and Interval Scheduling

Problem: given n activities, each with a start and end time, select the maximum number of activities that don't overlap (an activity can start exactly when another ends).

3.1 The greedy rule

Sort activities by end time, ascending. Repeatedly pick the next activity whose start time is $\geq$ the end time of the last picked activity.

```
int maxActivities(vector<pair<int,int>> &activities) { // {start,
end}
    sort(activities.begin(), activities.end(),
        [](auto &a, auto &b) { return a.second < b.second; }); // sort
    by end time
    int count = 0;
    int lastEnd = INT_MIN;
    for (auto &[start, end] : activities) {
        if (start >= lastEnd) {
            count++;
            lastEnd = end;
        }
    }
    return count;
}
```

3.2 Why sort by end time, not start time or duration?

This is the single most important intuition in this chapter, and a very common wrong instinct is to sort by duration (shortest activities first) or by start time. Neither is correct in general — a short activity in the middle of a packed schedule can still block more future activities than a longer one that ends early. Ending earliest leaves the most room for everything that comes after, regardless of how long the activity itself took.

3.3 The exchange argument for activity selection

Let OPT be any optimal solution, and let g be the activity greedy picks first (the one with the smallest end time overall). Let o be whatever activity OPT picks first. Since g has the earliest end time of any activity, $\text{end}(g) \leq \text{end}(o)$. Replacing o with g in OPT cannot break any other activity's compatibility, because every other activity in OPT already starts after $\text{end}(o)$, which is $\geq \text{end}(g)$ — so it also starts after $\text{end}(g)$. OPT is therefore still valid and still the same size after the swap. Repeating this argument for every subsequent pick shows greedy's solution can always be transformed into an equally-good optimal one, so greedy is optimal.

Complexity: $O(n \log n)$, dominated by the sort. Practice — dry run the greedy rule by hand on activities (1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10), (8,11), (8,12), (2,14), (12,16) sorted by end time, and confirm it selects 4 activities.

3.4 Variant: minimum number of meeting rooms

A related but different problem: given meeting intervals, find the minimum number of rooms needed so no two overlapping meetings share a room. This is not “pick the max non-overlapping subset” — every meeting must be scheduled somewhere.

```
int minMeetingRooms(vector<pair<int,int>> &meetings) {
    vector<int> starts, ends;
    for (auto &[s, e] : meetings) { starts.push_back(s);
    ends.push_back(e); }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());
    int rooms = 0, maxRooms = 0, i = 0, j = 0;
    while (i < (int)starts.size()) {
        if (starts[i] < ends[j]) { // a meeting starts before the
            earliest ongoing one ends
            rooms++;
            i++;
        } else {
            rooms--; // a room frees up
            j++;
        }
        maxRooms = max(maxRooms, rooms);
    }
    return maxRooms;
}
```

This is really a sweep over separated, sorted start and end events — the same flavor of technique as the sweep-line approach covered later in Advanced Algorithms. Complexity: $O(n \log n)$.

Practice — Chapter 3

Problem 14.1 — Maximum Non-Overlapping Intervals [Level 2]

Hint 1: This is activity selection with different wording.

Approach: Sort by end time; greedily pick an interval whenever it doesn't overlap the last picked one.

Complexity: $O(n \log n)$.

Problem 14.2 — Minimum Number of Arrows to Burst Balloons [Level 3]

Hint 1: An arrow shot at position x bursts every balloon whose range contains x .

Key observation: This is activity selection in disguise: sort by end coordinate, and greedily shoot at the end of the first not-yet-burst balloon, bursting every overlapping balloon at once.

Complexity: $O(n \log n)$.

Chapter 4 — Fractional Knapsack

Problem: given n items, each with a weight and a value, and a knapsack of capacity W , maximize total value — and unlike the 0/1 knapsack (covered later, in Dynamic Programming), you may take a fraction of an item.

In one sentence: sort items by value-per-unit-weight descending, and greedily take as much as possible of each item in that order until the knapsack is full.

```
double fractionalKnapsack(int W, vector<pair<int,int>> &items) {  
    // {weight, value}  
    sort(items.begin(), items.end(), [](auto &a, auto &b) {  
        return (double)a.second / a.first > (double)b.second / b.first;  
    });  
    // value/weight desc  
    double totalValue = 0;  
    int remaining = W;  
    for (auto &[weight, value] : items) {  
        if (remaining <= 0) break;  
        if (weight <= remaining) {  
            totalValue += value; // take the whole item  
            remaining -= weight;  
        } else {  
            totalValue += value * ((double)remaining / weight); // take a  
            fraction  
            remaining = 0;  
        }  
    }  
    return totalValue;  
}
```

Why does taking the best ratio first work? Because fractions are allowed, there is never a reason to leave room unused while a higher-ratio item is still available — swapping any amount of a lower-ratio item already taken for an equal weight of a higher-ratio item can only increase (or keep equal) the total value. This is exactly an exchange argument, applied to continuous amounts instead of discrete choices. Complexity: $O(n \log n)$.

⚠ Why this greedy rule fails for 0/1 knapsack

Without the ability to take fractions, the best-ratio item might not fit the remaining capacity at all, and greedily skipping it for a lower-ratio item that does fit can lock in a suboptimal total. The 0/1 version genuinely needs the subproblem exploration that dynamic programming provides — this exact contrast is revisited in Chapter 8.

Chapter 5 — Job Sequencing with Deadlines

Problem: given n jobs, each with a deadline and a profit, and each job taking exactly one unit of time on a single machine, schedule jobs to maximize total profit — a job only earns its profit if completed by its deadline.

In one sentence: sort jobs by profit descending, and greedily schedule each job as late as possible (the latest still-free time slot at or before its deadline).

```
int jobSequencing(vector<array<int,2>> &jobs) { // {deadline,
profit}
    sort(jobs.begin(), jobs.end(), [](auto &a, auto &b) {
        return a[1] > b[1]; // profit descending
    });
    int maxDeadline = 0;
    for (auto &j : jobs) maxDeadline = max(maxDeadline, j[0]);
    vector<bool> slotUsed(maxDeadline + 1, false);
    int totalProfit = 0;
    for (auto &[deadline, profit] : jobs) {
        for (int t = deadline; t >= 1; t--) { // latest free slot
            at/before deadline
            if (!slotUsed[t]) {
                slotUsed[t] = true;
                totalProfit += profit;
                break;
            }
        }
    }
    return totalProfit;
}
```

Why highest profit first? Every job we can fit in is worth including, so we should never let a lower-profit job occupy a slot that a higher-profit job could have used — processing profits in descending order guarantees each job gets the best slot still available to it before any lower-value job competes for the same slots. Why schedule as late as possible rather than as early as possible? Scheduling late preserves early slots for jobs with tighter (earlier) deadlines that haven't been processed yet — an early slot is more valuable because fewer jobs can use it.

Complexity: $O(n \log n + n \times \text{maxDeadline})$ with this simple slot scan; the inner scan can be improved to $O(n \alpha(n))$ with a DSU that tracks “the latest free slot at or before t ” directly — the same DSU structure from the Graphs Part, repurposed for a scheduling query instead of connectivity.

Practice — Chapter 5

Problem 14.3 — Job Sequencing to Maximize Profit [Level 3]

Hint 1: Every job takes the same amount of time (1 unit), which is what makes the slot-scan approach work directly.

Approach: Sort by profit descending; for each job, greedily place it in the latest available slot at or before its deadline.

Complexity: $O(n \log n + n \times \text{maxDeadline})$, or $O(n \alpha(n))$ with a DSU-based slot finder.

Chapter 6 — Merging Problems and the Huffman Pattern

Part XII (Heaps), Chapter 9 already introduced “Connect Ropes with Minimum Cost” as a heap application. This chapter names the underlying greedy idea properly: the Huffman pattern — always merge the two currently-cheapest items, because doing so minimizes the total accumulated merge cost.

```
long long minMergeCost(vector<int> &costs) {
    priority_queue<long long, vector<long long>, greater<long long>>
minHeap(
    costs.begin(), costs.end());
    long long totalCost = 0;
    while (minHeap.size() > 1) {
        long long a = minHeap.top(); minHeap.pop();
        long long b = minHeap.top(); minHeap.pop();
        long long merged = a + b;
        totalCost += merged;
        minHeap.push(merged);
    }
    return totalCost;
}
```

Why always the two cheapest? Every item's cost is counted once for every merge step it participates in — an item merged early gets added into the running total many more times than one merged late. To minimize the total, the cheapest items should be the ones that get “re-counted” the most, which means merging them first and merging the result (now more expensive) later. This is exactly the reasoning behind Huffman coding, which builds an optimal prefix-free binary code by repeatedly merging the two least-frequent symbols — the algorithm here is identical, only the interpretation of “cost” changes from “merge cost” to “encoded bit length $\times$ frequency.” Complexity: $O(n \log n)$.

Practice — Chapter 6

Problem 14.4 — Minimum Cost to Connect Sticks [Level 2]

Hint 1: Connecting two sticks costs the sum of their lengths, added to the running total.

Approach: Min-heap of stick lengths; repeatedly pop the two smallest, push their sum back, add it to the total cost.

Complexity: $O(n \log n)$.

Chapter 7 — More Classic Greedy Patterns

7.1 Gas station (circular feasibility)

Given gas and cost arrays around a circular route, find the starting station from which a full circular trip is possible, or determine none exists. Key observation: if the total gas across the whole circuit is less than the total cost, no starting point works at all. Otherwise, a solution is guaranteed to exist, and a single greedy pass finds it: track a running tank total starting from candidate station 0; whenever the tank goes negative at station i , no station from the current candidate up to i could have worked either, so reset the candidate to $i+1$ and the tank to 0.

```
int canCompleteCircuit(vector<int> &gas, vector<int> &cost) {
    long long totalTank = 0, currentTank = 0;
    int start = 0;
    for (int i = 0; i < (int)gas.size(); i++) {
        int diff = gas[i] - cost[i];
        totalTank += diff;
        currentTank += diff;
        if (currentTank < 0) {
            start = i + 1; // no station in [start, i] can be a valid start
            currentTank = 0;
        }
    }
    return totalTank >= 0 ? start : -1;
}
```

Complexity: $O(n)$, a single linear pass.

7.2 Jump game (reachability)

Given an array where $a[i]$ is the maximum jump length from index i , determine if the last index is reachable from index 0. Greedy rule: track the farthest index reachable so far; if the current index ever exceeds that farthest reach, the end is unreachable.

```
bool canJump(vector<int> &a) {
    int farthest = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        if (i > farthest) return false; // stuck before even reaching
        index i
        farthest = max(farthest, i + a[i]);
    }
    return true;
}
```

Complexity: $O(n)$.

7.3 Assign cookies / two-pointer matching greedy

A recurring shape: given two sorted arrays (e.g. children's greed factors and cookie sizes), match as many pairs as possible where one value satisfies a threshold on the other. Sorting both arrays and using two pointers — always trying to satisfy the most demanding remaining requirement with the smallest resource that still works — is a greedy pattern that shows up across scheduling, matching, and resource-allocation problems.

```
int findContentChildren(vector<int> &greed, vector<int> &cookies)
{
    sort(greed.begin(), greed.end());
    sort(cookies.begin(), cookies.end());
    int child = 0, cookie = 0, satisfied = 0;
    while (child < (int)greed.size() && cookie <
           (int)cookies.size()) {
        if (cookies[cookie] >= greed[child]) { // this cookie satisfies
        this child
            satisfied++;
            child++;
        }
        cookie++; // move to the next cookie regardless
    }
    return satisfied;
}
```

Complexity: $O(n \log n)$ for the sort, $O(n)$ for the two-pointer scan.

Practice — Chapter 7

Problem 14.5 — Jump Game II (minimum jumps) [Level 4]

Hint 1: Think in terms of “levels” reachable with a given number of jumps, similar to BFS layers.

Key observation: Within the range reachable by the current number of jumps, the best next jump is whichever index extends the farthest reach — greedily expanding one 'level' at a time.

Approach: Track current level's farthest boundary and the next level's farthest reach; increment the jump count each time you must cross the current boundary.

Complexity: $O(n)$.

Chapter 8 — When Greedy Fails

Greedy is only correct when the problem has the right structure (often called the greedy-choice property plus optimal substructure). This chapter exists because recognizing failure is just as important a skill as recognizing success.

8.1 Counterexample: coin change with arbitrary denominations

The greedy “always take the largest coin that fits” rule works for common currency systems (like $\{1, 5, 10, 25\}$) but fails for arbitrary denominations.

△ Concrete counterexample

Coins $\{1, 3, 4\}$, target = 6. Greedy takes 4 first (largest ≤ 6), leaving 2, which needs two more 1-coins: $4+1+1 = 3$ coins total. The optimal answer is $3+3 = 2$ coins. Greedy gives a worse answer here — this exact problem is solved correctly with dynamic programming (a later Part), not greedy.

8.2 Counterexample: 0/1 knapsack via ratio-greedy

Applying the fractional-knapsack ratio rule to 0/1 knapsack (no fractions allowed) can fail outright.

△ Concrete counterexample

Capacity $W = 50$, items given as (weight, value): A(10, 60) ratio 6, B(20, 100) ratio 5, C(30, 120) ratio 4. Ratio-greedy takes A then B (30 weight used, value 160), and C no longer fits (only 20 capacity left, C needs 30). But taking B and C together uses exactly 50 weight for value 220 — strictly better than greedy's 160. Committing fully to the best-ratio item locked in a worse combination.

8.3 Why these fail, in one sentence each

Problem	Why the greedy rule fails
Coin change, arbitrary denominations	A locally-largest coin can leave a remainder that needs more coins than a smaller first choice would have
0/1 knapsack via best ratio	Committing fully to the best-ratio item can waste capacity that a different combination would use better
Longest path in a general graph	“Always take the longest available edge” has no exchange argument backing it — no known efficient algorithm solves this problem at all (it's NP-hard)

The pattern to take away: greedy fails whenever an early locally-best choice can close off a globally-better combination that wasn't visible yet. Problems where this can happen usually need dynamic programming, which explicitly keeps track of multiple possible partial solutions instead of committing early.

Chapter 9 — How to Recognize a Greedy Problem

Clue	Likely greedy pattern
“Maximum number of non-overlapping intervals”	Activity selection — sort by end time

Clue	Likely greedy pattern
“Minimum number of rooms/resources for overlapping intervals”	Sweep over sorted start/end events
“Maximize value, items divisible/fractional allowed”	Fractional knapsack — sort by value/weight ratio
“Maximize profit, deadline per job, unit time per job”	Job sequencing — sort by profit, place as late as possible
“Minimum total cost to repeatedly combine/merge items”	Huffman pattern — min-heap, always merge the two cheapest
“Can you always reach the end, given per-step limits”	Track farthest reachable index in one linear pass
“Match two sorted groups optimally under a threshold”	Two-pointer greedy on both sorted arrays

✦ The one habit that matters most

Before trusting any greedy idea, try to state its exchange argument in one or two sentences. If you can't explain why swapping any optimal solution's choice for greedy's choice can't make things worse, actively hunt for a counterexample (small, hand-built cases like Chapter 8's are usually enough to break a wrong greedy rule) before coding it.

Part XIV — Master Revision

Must Remember

- Greedy makes one irreversible locally-best choice per step; it is only correct when the problem's structure guarantees this doesn't cost global optimality.
- An exchange argument (transform any optimal solution into greedy's solution without making it worse) is the standard way to prove a greedy rule correct.
- Activity selection sorts by END time, not start time or duration — ending earliest leaves the most room for everything after.
- Fractional knapsack sorts by value-per-weight ratio; 0/1 knapsack (no fractions) needs dynamic programming instead — the ratio rule can fail there.
- Job sequencing with deadlines: sort by profit descending, place each job in the latest free slot at or before its deadline.
- The Huffman/merging pattern: always merge the two currently-cheapest items via a min-heap, because early items get “re-counted” more often.
- “Passes a few hand-tried examples” is not a proof — always look for either an exchange argument or a counterexample.

Common Mistakes Recap

Mistake	Fix
Sorting activities by duration or start time instead of end time	Always sort by end time for activity selection
Applying fractional-knapsack ratio greedy to 0/1 knapsack	Use dynamic programming for 0/1 knapsack instead
Trusting greedy coin change on arbitrary denominations	Verify the denomination system first, or use DP for arbitrary coins
Scheduling jobs earliest-first instead of latest-first in job sequencing	Place each job as LATE as possible to preserve early slots for tighter deadlines

Mistake	Fix
Assuming a greedy idea is correct because it worked on 2-3 examples	Build a small adversarial example or find the exchange argument before trusting it

Complexity Table — Greedy Patterns

Pattern	Complexity
Activity selection / interval scheduling	$O(n \log n)$
Minimum meeting rooms	$O(n \log n)$
Fractional knapsack	$O(n \log n)$
Job sequencing with deadlines (simple slot scan)	$O(n \log n + n \times \text{maxDeadline})$
Huffman / minimum merge cost	$O(n \log n)$
Gas station feasibility	$O(n)$
Jump game reachability	$O(n)$

Interview-Style Questions

- 1 What is an exchange argument, and why does it prove a greedy algorithm optimal?
- 2 Why does activity selection sort by end time instead of start time or duration?
- 3 Why does the fractional-knapsack ratio-greedy rule fail for the 0/1 knapsack problem?
- 4 Why does job sequencing with deadlines place each job as late as possible rather than as early as possible?
- 5 Why does the Huffman merging pattern always combine the two cheapest available items?
- 6 Give an example where a greedy strategy looks correct on small hand-tried cases but fails in general.

Cheat Sheet — Part XIV

```
// Activity selection: sort by END time ascending, greedily pick
if start >= lastEnd
// Fractional knapsack: sort by value/weight DESCENDING, fill
greedily
// Job sequencing: sort by profit DESCENDING, place in latest
free slot <= deadline
// Merge/Huffman pattern: min-heap, repeatedly merge the two
smallest, sum into total
// Gas station: running tank total; reset candidate start
whenever tank goes negative
// Jump game: track farthest reachable index in one linear pass

// Before trusting ANY greedy idea:
// 1) Can I state an exchange argument in 1-2 sentences?
// 2) Can I build a small counterexample that breaks it?
// If neither is clear, it probably needs Dynamic Programming
instead.
```

Coming up in Part XV

Part XV — Dynamic Programming is one of the largest sections of this book. It starts exactly where this Part's failures left off: coin change and 0/1 knapsack, both broken for greedy, become the first two worked examples of state, transition, and the memoization-vs-tabulation choice — the thinking process that carries through 1D/2D DP, string DP, tree DP, and bitmask DP.